

Paged Attention

yuro-py.github.io

23 September 2026

Abstract

KV-cache grows unbounded as context length grows, and naive contiguous allotment for it wastes memory through internal/external fragmentations, limiting the number of requests a GPU can serve at once. Paged Attention borrows the operating system's paging idea, splitting the KV-cache into fixed-length blocks and mapping them through a block table, so memory is allotted just-in-time instead of reserved upfront. This note summarizes the KV-cache bottleneck, the fragmentation it causes, and how vLLM's block manager and block-wise attention computation eradicate it and boost throughput.

1 Introduction

Pre-training happens in parallel, compute-bound workloads because it has to compute massive matmuls upon entire batches, meanwhile inference is sequential, and memory-bound. KV-cache is a major bottleneck, which carries necessary contextual information for next token prediction, but is difficult to manage and wastes a lot of memory and causes internal/external fragmentations, which are reduced significantly in vLLM through Paged Attention, and boosts throughput.

2 Training vs Inference

1. Training is when a model learns from the dataset, makes predictions for all positions in the sequence parallelly, calculates loss, backprops and optimizes the model. GPUs are well suited for this parallelism and, during training they are fully saturated, and increasing the compute power will speed up training by introducing even more parallelism, AKA compute-bound, with memory not being a significant limiting component.
2. Inference on the other hand, is memory bound. It just does next-token prediction because masking doesn't permit to view future tokens so massive parallel compute requirement is significantly reduced. During inference, it's dominated by repeated memory accessing, loading model weights, and caching keys and values.

3 Two Stages of Inference for a Single Request

1. **Prefill** → It runs the forward pass upon the entire input prompt, gives us the key-value pairs for each prompt for every attention layer, and generates the first output token, measured

in TTFT (or Time To First Token). This step is closer to pre-training in nature because it requires massive parallel matrix multiplications.

2. **Decode** → After the first output token, the remaining output tokens are generated by decode step, and measured in TPOT (or Time Per Output Token) or ITL (Inter Token Latency). This is where the majority of inference workload lies in, assuming we are dealing with longer context and outputs.

4 KV-Cache

Every attention layer generates three projections for every token: Query, Key, Value. To generate the next token, we need the query of the recently generated token, and key-value pairs of recent+all-past tokens. As we move forward into the output length, recomputing every past key-value pair is computationally expensive and hence we cache it right after generation and reuse it for every new token, which saves a lot of computational overhead. As the context grows, even storing the KV-cache in raw memory becomes expensive eventually.

There have been other forms of attention that significantly reduce the KV-cache in different forms of attention:

- a. **GQA** → Inspired from MHA, it divides the query into heads, but divides the key-values into “lesser” number of heads, groups them and broadcasts them to all query heads.
- b. **MQA** → It goes even further to just have one key-value head broadcasted to full heads of query, for one single token.
- c. **MLA** → Introduces “low-rank key-value joint compression” which caches the key-value pairs into a lower dimensional latent space.

5 Naive KV-Caching Introduces Fragmentations

- a. Storing KV-cache in GPU, unorganized, wastes a lot of memory because it stores per-token cache collectively in a contiguous manner inside GPU memory, and a lot of reserved space is unused if the output token is shorter than predicted meanwhile the next request ignores it, this is called “Internal fragmentation.”
- b. Internal fragmentation when stacks up, introduces “External Fragmentation,” where even though there’s enough memory to store the next request’s key-value pairs, but it’s not contiguous anymore because of lots of internal fragmentations in memory.

6 Paged KV-Caching

vLLM introduces Paged Attention where internal/external fragmentations are almost significantly eradicated by taking inspiration from operating system’s paging concept, and operating on “just in-time block allocation” instead of massive memory allotment beforehand.

The main component is called a “KV Cache Manager” which is responsible for a lot of tasks. It organizes GPU memory into the virtual “Logical Memory,” and “Physical Memory,” and uses “blocks”

for storing tokens, instead of a contiguous large buffer per sequence. Blocks have a fixed pre-defined length for tokens.

The Logical Block IDs are the order in which the tokens processed sequentially. Physical Memory has allocated/unallocated KV blocks. Block Table maps the former with the latter, with these details:

- a. block-id $\rightarrow$ id of the blocks.
- b. reference count $\rightarrow$ number of requests currently accessing this block.
- c. block hash $\rightarrow$ used in prefix caching.

When the existing allocated block pool runs out of slots, KV-cache manager accesses the unallocated “global free block pool” through a doubly linked list and updates the block table with the new mapping, which is “appending new block” from our point of view, and when the request is done for, KV-cache manager removes it from block table and adds its address back to the global free block pool so some other request can use it.

Flow

Tokens per Block defined, or Block Length = 6.

- a. REQUEST-1’s prompt has 10 tokens (eg: “hello there how are you I am fine thank you”). The key-value pairs for these ten tokens will be stored in $\rightarrow \lceil 10/6 \rceil = 2$ blocks (BLOCK-ID 1 and 2, therefore $6 + 6 = 12$ slots), with 2 slots left empty for next execution step’s storing of a request/output token.
- b. REQUEST-2’s prompt has 6 tokens so it takes up the final two slots in BLOCK-2, then KV-CACHE manager brings an empty block from the block pool of id = 3, of default length = 6 and appends it to physical memory, then stores the rest 4 tokens of REQUEST-2’s tokens in BLOCK-3.
- c. RESPONSE-1 $\rightarrow$ The next execution step is where the response for REQUEST-1 is getting generated and it would have 12 tokens, so we will continue allocating the first two tokens in the leftover slots of BLOCK-3, and then KV-CACHE manager brings $\lceil 10/6 \rceil = 2$ blocks (4 and 5) and allocate the leftover 10 tokens in these blocks meanwhile everything gets mapped into the block table.

block 1 = 6
 block 2 = 4 + 2
 block 3 = 4 + 2
 block 4 = 6
 block 5 = 4

Block 2 and block 3 got a reference count of 2, because they are accessed by two different requests.

One thing implied through this flow is that it doesn’t matter if we store “two requests” side by side instead of a “request and response” as traditionally expected side by side, because everything is easily

traecable through the block table when any of those kv-pairs are actually needed. The blocks in nature are read-shared and write-unique for corresponding requests.

If two requests got the same tokens, meaning their kv-cache is same, they continue to use same until their common prefix breaks with a unique token for each request. The kv-cache manager does “copy on write” and copies request A’s kv-cache to a new fresh block and request B continues in the same block.

7 Paged Attention

Paged Attention accesses the required keys and values in their non-contagious organization from the block table’s sequential mapping through logical memory, and performs attention.

We gather all positions of cached keys and values separately. First, block IDs in $J = 1, 2, 3, 4 \dots$

Keys and values in different blocks:

$$K_j = (k_{(j-1)B+1}, \dots, k_{jB})$$

$$V_j = (v_{(j-1)B+1}, \dots, v_{jB})$$

Compute block-wise attention, apply running softmax normalizer to get weights then multiply with values block-wise and sum it up. The math is identical to regular attention.

$$A_{ij} = \frac{\exp(q_i^\top K_j / \sqrt{d})}{\sum_{t=1}^{\lceil i/B \rceil} \exp(q_i^\top K_t / \sqrt{d})},$$

$$o_i = \sum_{j=1}^{\lceil i/B \rceil} V_j A_{ij}.$$

Figure 1: Block-wise Paged Attention computation.

References

- [1] Kwon, W. et al., “Efficient Memory Management for Large Language Model Serving with PagedAttention,” *arXiv preprint arXiv:2309.06180*, 2023. Available at: <https://arxiv.org/pdf/2309.06180>.
- [2] Elshafie, H., “Paged Attention from First Principles: A View Inside vLLM.” Available at: <https://hamzaelshafie.bearblog.dev/paged-attention-from-first-principles-a-view-inside-vllm/>.